

Top 10 MCP Interview Questions

For every AI Role



1. What does the 'P' in MCP stand for?

- 'P' stands for Protocol, which refers to a network protocol. Common examples include TCP/IP and HTTP, which are types of network protocols.
- Network protocols define how two parties establish a connection, send data, define data formats, and handle errors.
- MCP is a network protocol that defines the network communication data format for integrating AI models with external data sources and tools, including the transmission protocol, data format definition, and authentication methods.

2. What problem does MCP solve?

- MCP addresses the fragmentation issue of integrating large language models with external data sources and tools. By providing a standardized interface, MCP enables AI applications to connect more reliably and efficiently to different data sources and tools, avoiding the hassle of developing a connector for each data source.
- Simply put, a tool using the MCP protocol can be used by both Claude and cursor without needing to be developed twice.



3. How does MCP differ from previous OpenAI plugins?

- Earlier OpenAI plugins required custom implementations for each tool—every plugin had its own API schema, authentication, and platform-specific logic. This created fragmentation and made reuse difficult. MCP solves this by defining a universal interface for tools, resources, and prompts that any LLM client can interact with using the same protocol.
- MCP uses a client–server architecture, where tools are exposed through MCP servers. Any AI model that supports MCP can automatically discover and use these tools without additional integration work. This eliminates the old “one plugin per platform” problem and replaces it with write once, use anywhere.
- Another key difference is that MCP supports stateful, multi-step, agentic workflows, not just simple one-shot function calls. This makes it more suitable for real agent systems that need to chain tools, maintain context, and perform complex operations.



4. Is MCP suitable for agents?

- MCP is not designed for agents, it is designed for models to connect with external resources and tools.
- Several issues remain unresolved, such as MCP's centralized identity authentication technology, which is not suitable for agent-to-agent authentication. MCP's CS architecture (client-server architecture) means the server cannot actively connect to the client.

5. Can you explain the fundamental difference between A2A and MCP protocols?

- A2A (Agent-to-Agent) and MCP (Model Context Protocol) serve different primary functions. A2A, developed by Google, focuses on standardizing communication between AI agents built on different frameworks, enabling them to collaborate, discover capabilities, and share context.
- MCP, created by Anthropic, standardizes how LLMs access external data, tools, and dynamic context. In simple terms, A2A is about agent-to-agent communication, while MCP is about model-to-resource interaction.



6. What are the core components of MCP's architecture?

MCP's architecture consists of four main components:

1. MCP Host: An LLM-based application that needs to access external resources
2. MCP Server: A service that exposes data, tools, and prompts to the host
3. MCP Client: The connector that facilitates communication between host and server
4. Resources: These include data assets (app-controlled), tools (API/functions that are model-controlled), and prompts (context instructions that are user-controlled)

7. What is the difference between MCP and function calling?

- Function calling is the interaction method between large models and the external digital world. MCP is the interaction method between the MCP host (chatbot or AI tool) and external tools/resources.
- Generally, a model first triggers a function call through function calling, which then triggers an MCP request within that function call.



8. How does MCP handle tool execution, and what are its limitations?

In MCP, tools are APIs or functions that the model can call as needed. The MCP Server exposes these tools to the MCP Host (the LLM application), which can then execute them based on the model's decisions. However, MCP's approach to tool execution has several limitations:

1. No built-in state management between tool calls
2. Limited support for long-running or asynchronous tasks
3. No native mechanism for tool-to-tool communication
4. Less robust security controls around tool execution
5. No standardized way to negotiate output formats across different systems

9. Are MCPs only for LLMs or can they integrate with rules engines?

- No, MCPs aren't only for LLMs. MCP is a general protocol, so any system—including rules engines—can use it. A rules engine can act as an MCP client to call tools or as an MCP server to expose its own rule-evaluation capabilities.



10. What are the steps to implement an MCP Server to expose a custom tool?

Implementing an MCP Server to expose a custom tool requires:

1. Define the tool's interface, including inputs, outputs, and expected behavior
2. Create an MCP resource definition that describes the tool, including its parameters and return types
3. Implement the actual tool functionality (the code that will execute when called)
4. Build an MCP Server that can receive requests, validate them, and route them to the appropriate tool implementation
5. Implement authentication and authorization logic if needed
6. Set up the necessary infrastructure for the MCP Server, including networking, scaling, and monitoring
7. Publish documentation about the tool so MCP Hosts know how to interact with it
8. Test the integration with various MCP Clients to ensure compatibility



LIKE THIS CONTENT?

FOLLOW FOR MORE!



LIKE

COMMENT

SHARE

SAVE